Bashir Agro System
Phase 12A V2: Admin Web Field Boundary Management + Laravel API + PostGIS Implementation

PHASE 12A V2 OBJECTIVE

This phase upgrades the location system from simple latitude/longitude radius into a real agricultural field boundary system.

Main workflow:

Admin creates farmer

↓

Admin draws actual farm boundary on satellite map

↓

Laravel stores polygon using PostGIS

↓

API sends field information to mobile app

↓

Backend verifies farmer GPS inside field boundary

This phase is mainly:

- Laravel Admin Panel
- Backend API
- PostgreSQL + PostGIS

Flutter implementation is Phase 12B V2.

TECHNOLOGY STACK

Admin Web:

- Laravel Blade
- JavaScript
- Mapbox GL JS

- Mapbox GL Draw
- Turf.js

Backend:

- Laravel
- REST API
- Sanctum

Database:

- PostgreSQL
- PostGIS

Data:

- GeoJSON
- Polygon Geometry

NEW ARCHITECTURE

Old:

Farmer
|
Latitude
|
Longitude
|
Radius

New:

Farmer
|
Farm Location
|
Polygon Boundary
|
Task Assignment
|
GPS Verification

ADMIN PANEL WORKFLOW

Modify:

+ [redacted]

Add:

Farm Information

Fields:

- Farm Name
- Farm Type
- Address
- Field Boundary
- Area

Process:

1. Admin enters farmer information.

2. Opens map.

3. Searches farm location.

4. Draws polygon around field.

5. System calculates area.

6. Saves boundary.

MAP IMPLEMENTATION

Use:

Mapbox GL JS

Mapbox GL Draw

Turf.js

Required features:

- Satellite view
- Search
- Polygon drawing
- Polygon editing
- Delete polygon

- Area calculation

DATABASE DESIGN

Create:

farm_locations

Fields:

id

farmer_id

name

address

boundary

area

created_by

status

created_at

updated_at

Boundary type:

geometry(POLYGON,4326)

POSTGIS SETUP

Enable:

CREATE EXTENSION postgis;

Purpose:

- Store field polygons
- Calculate spatial relationships
- Validate GPS position

GEOJSON FLOW

Admin draws polygon.

Map returns GeoJSON.

Laravel receives GeoJSON.

Laravel converts it into PostGIS geometry.

Database stores polygon.

API REQUIREMENTS

Create:

GET /api/v1/farms

Returns farmer farms.

GET /api/v1/farms/{id}

Returns:

- Farm name
- Boundary
- Area

POST /api/v1/farms

Create farm boundary.

PUT /api/v1/farms/{id}

Update boundary.

TASK RELATIONSHIP

Final structure:

Farmer

|

Farm Location

|

Task Assignment

|

Task Submission

Example:

Farmer:
Rahim

Field:
North Rice Field

Task:
Apply Fertilizer

GPS VERIFICATION

Farmer submits:

- latitude
- longitude
- accuracy

Laravel creates GPS point.

PostGIS checks:

Is GPS point inside polygon?

TRUE:

Accept submission

FALSE:

Reject or flag

SECURITY

Do not trust mobile GPS alone.

Backend is the final authority.

Validate:

- Permission
- Assignment ownership
- GPS accuracy
- Submission timing
- File uploads

TESTING

Admin:

✓ Create farmer

✓ Draw boundary

✓ Save polygon

✓ Edit boundary

Database:

✓ Polygon stored

✓ Area calculated

API:

✓ Return GeoJSON

✓ Authentication works

GPS:

✓ Inside accepted

✓ Outside rejected

ANTIGRAVITY MASTER INSTRUCTION

Implement PHASE 12A V2 ONLY.

Goal:

Build field boundary management for Bashir Agro Admin Panel.

Implement:

1. Mapbox satellite map integration.

2. Polygon drawing and editing.

3. GeoJSON capture.

4. PostGIS polygon storage.

5. Farmer and farm relationship.

6. Farm boundary APIs.

7. Spatial validation services.

Do not implement Flutter app.

Do not create mobile screens.

Preserve:

- Existing farmer CRUD
- Existing authentication
- Existing task modules

After implementation report:

- Files created
- Files modified
- Database migrations
- API endpoints
- PostGIS changes
- Testing results

Run:

php artisan test

vendor/bin/pint

Stop after Phase 12A V2 completion.